Robust Insertion of Deformable Linear Objects Using Recursive Expert-Initialized Deep Reinforcement Learning

Anonymous Authors

***Abstract*— Manipulation of Deformable Linear Objects (DLOs) remains a challenge in robotics due to their high-dimensional configuration space, underactuation, and dependence on physical parameters such as elasticity and length. As a result, generalization across DLOs is a limitation of existing DLO insertion methods. This paper presents a recursive expert-initialized reinforcement learning framework for DLO insertion that enables use of sparse rewards and integrates Rapid Motor Adaptation (RMA) to generalize across task parameters. We show that access to latent DLO parameters has a larger impact on insertion performance than the choice of reinforcement learning algorithm, and that recursive expert-initialized reinforcement learning enables training of policies using sparse rewards over a large domain of task parameters. Simulation results demonstrate improved DLO insertion success rates from 89% to 98% compared to a previous method. Zero-shot transfer to real-world DLO insertion tasks is achieved with a success rate of 100% for nine test cases.**

I. Introduction

Deformable Linear Object (DLO) such as cables and sealings are integrated into many products, but current industrial automation strategies for handling and installing such objects are expensive, rigid, and inflexible [1], [2].

DLO manipulation requires addressing two problems: 1) DLO state estimation; 2) DLO manipulation. Problem 1 mainly poses a challenge due to the inherent deformability, which complicates state estimation since no CAD model is available. Problem 2 is challenging since the system is underactuated and due to the non-collocated nature of DLO manipulation. Computer vision is typically used for estimating the state of DLOs; in this domain model-free methods and model-based methods have been proposed. Typically, these methods produce inaccurate estimates when parts of the DLO are occluded. DLO manipulation can be categorized into four different tasks [3]: Knot tying, insertion, untangling, and shape control. Of these, this work will address the insertion task. Work in solving insertion of DLOs covers a wide range of methods; [4] leverages a local pose estimate from tactile sensors, followed by a simple insert, made possible by the grasp being located near the tip.

For more complex manipulation, reinforcement learning (RL) and visual cues [5] have been used for DLO insertion in simulation, and consequently, building upon the work by adding state-of-the-art behaviour cloning methods in [6], which used Diffusion Policy (DP) [7] to train the deep reinforcement learning (DRL). A model-based approach [8] uses a simulation model to optimize camera pose for high-precision inserts. For a more general approach [9] has applied the increasingly popular Vision-Language-Action (VLA)-based methods to simpler problems, and demonstrates the utility of incorporating long-horizon actions.

As the complexity of control and planning problems increases, learning-based methods have become increasingly attractive approaches to address Problem 2, demonstrating the capability to control high-dimensional systems in real-time successfully. In this domain, especially three categories of methods have prevailed: imitation learning (IL) based methods, foundation model-based methods, and DRL. While each solution has its strengths and weaknesses, hybrid systems that combine these methods have garnered increased attention due to their ability to leverage the strengths of each method while mitigating their weaknesses [10]. These methods include expert-initialized DRL, where expert data is injected into the replay buffer of the agent [11], [12], methods where trained IL policies act as references for DRL [13], [14], and methods where humans-in-the-loop guide the learning process [15], which demonstrate the power of sparse reward design.

In this work, we present a learning framework for DLO insertion. The framework is based on the expert-initialized reinforcement learning algorithm Deep Reinforcement Learning with Reference (DRLR), but instead of using a fixed reference policy, the learning algorithm is run recursively with increased domain randomization and refined reference policies. The framework is combined with Rapid Motor Adaptation (RMA) for learning a latent space representation of task parameters. For deployment in a real setup, we use global DLO state estimation using a multi-view DLO tracking algorithm. We present the following contributions

- A recursive expert-initialized reinforcement learning framework that recursively promotes learned policies to reference policies as domain randomization is increased.
- Integration of Rapid Motor Adaptation into DLO insertion for inference of latent task parameters.
- A simulation-based analysis showing that information about latent DLO properties is more important for DLO insertion than the underlying RL algorithm.
- Zero-shot transfer of policy to real-world DLO insertion.

This work is structured by Sec. II presenting the related work, Sec. III formulates the problem, Sec. IV presents the proposed framework, Sec. V provides DLO insertion specific detains and Sec. VI shows the results. Finally Sec. VII concludes on this work.

All authors are with the Maersk Mc-Kinney Moller Institute, University of Southern Denmark, Denmark {vims, chsl}@mmmi.sdu.dk

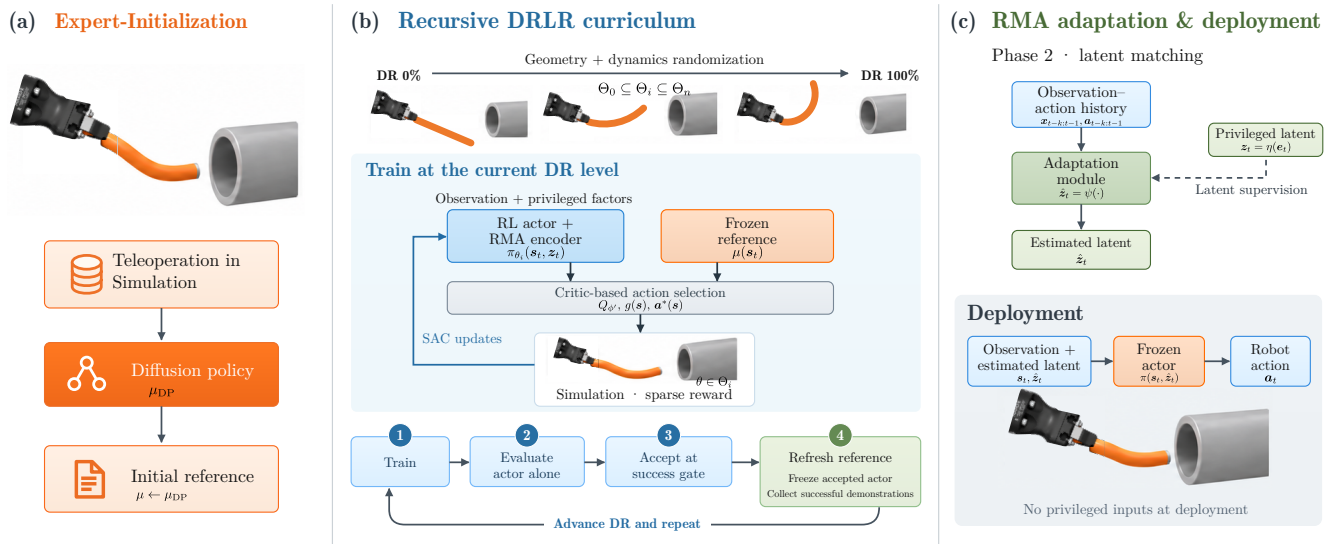


Fig. 1: **Overview of the DRLR–RMA training and deployment pipeline**. (a) Simulation demonstrations initialize a diffusion policy that serves as the initial reference. (b) The actor is trained under progressively increased geometry and dynamics randomization, using critic-based action selection between the actor and a frozen reference. Once the actor alone meets the success criterion, the reference is refreshed and the curriculum advances. (c) The RMA adaptation module learns to estimate privileged latent information from observation–action history. During deployment, the frozen actor uses observations and the estimated latent to control the gripper without privileged inputs.

II. Related Work

This work applies expert-initialized DRL for DLO manipulation; the related work touches on these, but for DLO manipulation, DLO perception is required and therefore covered first. Finally, methods for bridging the sim2real gap are presented.

DLO perception involves estimating the state, global vision-based methods using splines [16], self-supervised learning [17], and diffusion models [18], all provide the state as a set of 3D points connected with piecewise linear (PWL) segments. Alternatively, for 2D DLO routing [19] uses a spatial state representation while [20] uses ResNet feature encoders in a hierarchical imitation learning framework. While each provides impressive cycle times and robustness, they still fail in handling tip occlusion, which is an essential attribute in common DLO manipulation tasks like insertion. This problem was addressed in [21] and [22], which applies motion coherence theory to track both the DLO body and tips even through occlusions, while [23] attempts to build on these works using Gaussian-Splatting.

DLO manipulation, as addressed in [10], presents three overarching categories: The analytical approach, the reinforcement learning (RL) approach, and the imitation learning (IL) approach. Of these, the methods within the RL category have shown great potential in their capabilities to manipulate DLOs using both physics information and expert demonstrations [10]. While work in reinforcement learning without these has enabled [24] to train DRL policies to solve tasks in agents with high degrees of freedom (DOF) across multiple tasks, the reward shaping required and training time are both extensive. [6] instead uses DRL with Domain Randomization (DR) and motion primitives to generalize policies, and uses these to generate rollouts used to train a DP model, which additionally adds the necessity to design said motion primitives. Instead of using primitives, expert-initialized RL provides strong results for complex, long-horizon, and multi-stage tasks [10] and is therefore the chosen paradigm in this work. **Expert-initialized reinforcement learning**, as addressed in [25], provides a tree of method groupings, reflecting how the expert information is integrated into the learning process. Of the groups, prior knowledge with pre-training has shown outstanding results [25] for complex task solving. Within this group, different approaches like [11], [12] inject expert data into the replay buffer, while [26] uses the demonstrations to augment the policy gradient for improved convergence. These, however, suffer from the models forgetting the expert behaviour once online transitions dominate the replay buffer. In order to counteract this, expert policies have been integrated directly into the learning process. These methods contain trained IL policies, which act as references for DRL [13], [14]. These reference-based learning paradigms come with the attractive trait of enabling sparse reward design, which significantly reduces the otherwise labor-intensive process of reward shaping. While all these methods operate in simulation, work in learning from a real setup [27], has also shown impressive results with humans-in-the-loop guiding the learning process [15]. Successful transfer of simulation-trained policies to physical systems requires mitigating the **sim2real** gap. To address the gap, DR [28], [29], [30] has become a common first approach to training a policy in simulation on a range of physical parameters. Continuous Domain Randomization

(CDR) applies this but in a gradual manner to improve convergence [31]. System Identification (SysId), by contrast, explicitly estimates parameters of the physical system and incorporates these estimates into the control or policy framework [32], [33]. Related adaptation methods avoid the explicit estimation of the dynamical parameters, and instead infer them from a learned latent space representation based on a history of previous actions and observations. One such example is RMA, which has shown zero-shot transfer in quadruped locomotion [34], bipedal locomotion [35], and dexterous manipulation [36].

III. Problem Formulation

The objective of this work is to enable a robot to insert a DLO through a hollow cylinder, as illustrated in Fig. 2. The insertion strategy should be applicable to a variety of DLOs and cylinder configurations.

A. Task Definition

The task is defined by the geometry of the cylinder and the DLO, where the cylinder is represented by the tuple (c, ρ, n, L) where $c \in \mathbb{R}^3$ is the center of the entry hole of the cylinder with radius $\rho > 0$, $n \in \mathbb{R}^3$ is the inwards pointing unit normal vector at point c, and L is the length of the cylinder. The interior of the cylinder is denoted by $\Omega \subset \mathbb{R}^3$.

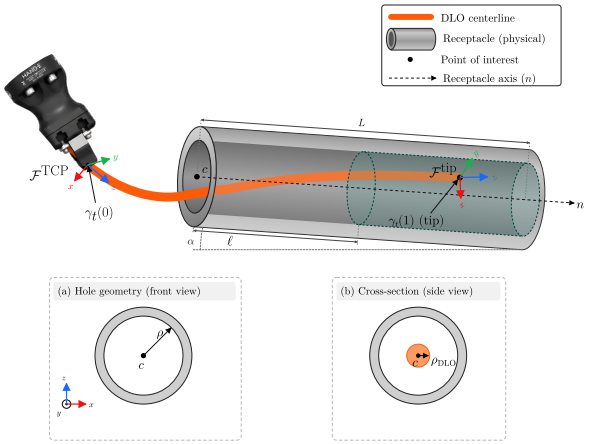


Fig. 2: DLO insertion task specification. The DLO is represented by its centerline γ_t, with grasp point $\gamma_t(0)$ and tip $\gamma_t(1)$. The DLO is inserted into a cylinder with length L and radius $\rho > \rho_{\text{DLO}}$.

The DLO has length ℓ_{DLO} and is represented by the position of its centerline at time t

$$\gamma_t : [0, 1] \to \mathbb{R}^3, \tag{1}$$

with $\gamma_t(1)$ being the tip position. The DLO is assumed to occupy the tubular neighborhood of radius $\rho_{\text{DLO}} > 0$ around the centerline. The robot action is applied through a frame $\mathcal{F}_g(t) \in \text{SE}(3)$ associated with the grasp point on the centerline $\gamma_t(0)$.

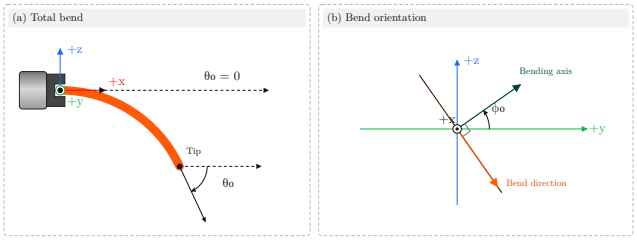


Fig. 3: Initial-curvature parametrization of the cable. (a) The total bend angle θ_0 measures the change in tangent direction along the cable. (b) The azimuth ϕ_0, measured from local $+y$ toward $+z$, defines the bending axis in the cross-sectional plane; the bend direction is perpendicular to this axis.

We say that the insertion task is completed when the tip position of the DLO $\gamma_t(1)$ is inserted a distance $\ell \in (0, L)$ into the cylinder, i.e.

$$\gamma_t(1) \in \{p \in \Omega \mid n^T(p - c) \geq l\} =: \Omega_{in}. \tag{2}$$

For convenience, a coordinate frame $\mathcal{F}^{\text{tip}}$ is attached to the DLO tip with z-axis denoted by $\boldsymbol{z}^{\text{tip}} \in \mathbb{R}^3$.

B. Task Parameterization

The insertion task is parameterized by the physical properties of the DLO, the cylinder and their interaction, the initial DLO configuration, and observation uncertainty, see a full list of parameters and their considered ranges in TABLE IV. The task parameters are denoted by

$$\theta = (\theta_{\text{DLO}}, \theta_{\text{env}}, \theta_{\text{init}}, \theta_{\text{obs}}), \tag{3}$$

where θ_{obs} represents measurement noise on observations and the DLO parameters are

$$\theta_{\text{DLO}} = (\rho_{\text{DLO}}, \ell_{\text{DLO}}, E, G, \lambda, c_b), \tag{4}$$

where ρ_{DLO} and ℓ_{DLO} denote the DLO radius and length, E and G denote the Young's and shear moduli, λ is the density, and c_b is the bending damping coefficient.

The environment parameters are

$$\theta_{\text{env}} = (\mu, \rho, l, \alpha), \tag{5}$$

where μ denotes the DLO–cylinder sliding friction coefficient, $\rho > \rho_{DLO}$ is the radius of the cylinder, l is the insertion depth, and α denotes the angle of the cylinder.

The initial DLO configuration is parameterized by

$$\theta_{\text{init}} = (\theta_0, \phi_0), \tag{6}$$

where θ_0 denotes the initial-curvature of the DLO and ϕ_0 its direction. The DLO initially has constant curvature θ_0 and ϕ_0 determines the angle about the z-axis of frame $\mathcal{F}_g(t)$ and the direction of the curvature, as shown in Fig. 3.

The task parameters are assumed to be unknown during the execution of the DLO insertion task. It is however assumed that the centerline of the DLO is observed.

IV. Method

A. Overview of Proposed Framework

The proposed recursive expert-initialized reinforcement learning algorithm is illustrated in Fig. 1 and consists of a) expert initialization to enable the use of sparse rewards, b) recursive expert-initialized DRL to allow large domain randomization, and c) RMA to adapt the learned policy the task parameters. Existing expert-initialized DRL algorithms rely on a fixed expert reference policy throughout the learning. However, in this work the reference policy is updated after each cycle of the algorithm, before increasing the domain randomization.

It should be noted that all data used for determining the initial policy and learning the reinforcement learning policy is generated in simulation.

B. Collection of Demonstrations and Generation of Reference Policy

To enable reinforcement learning with a sparse reward, the reinforcement learning is initialized with a reference policy that is capable of completing the considered task for a fixed value of the task parameters as shown in Fig. 1(a). It is chosen to generate a diffusion policy as the initial policy and train it based on expert demonstrations using the following procedure

1) Collect 70 demonstrations that successfully solve the task by teleoperation in simulation.
2) Train a DP, μ_{DP}, on the demonstrations and evaluate it over 512 rollouts, requiring at least an 80% success rate before proceeding.
3) Collect successful rollouts of the reference policy and store their transitions in a demonstration buffer $\mathcal{D}$.

The demonstration buffer is defined as

$$\mathcal{D} = \{(s_1^i, a_1^i, z_1^i), \ldots, (s_{N_i}^i, a_{N_i}^i, z_{N_i}^i)\}_{i=1}^{n_r} \tag{7}$$

where (s_t^i, a_t^i, z_t^i) is the state, action, and latent representation of the privileged parameters at time t for rollout i, N_i is the length of rollout i, and n_r is the number of rollouts.

C. Recursive Expert-Initialized Deep Reinforcement Learning

A main issue in the considered task is that changes in the task parameters significantly change the robot behavior required to successfully complete the task. Consequently, it is proposed to gradually increase the parameter range used for domain randomization instead of only training with the full parameter range. This is accomplished by the proposed recursive expert-initialized DRL that reinitializes the reinforcement learning with the reference policy from the previous iteration instead of using the initial expert-initialized strategy throughout the learning.

The proposed recursive curriculum learning is presented in Algorithm 1 that relies on an ordered curriculum $\{\Theta_i\}_{i=0}^n$ defined by parameter sets

$$\Theta_0 \subseteq \Theta_1 \subseteq \cdots \subseteq \Theta_n \tag{8}$$

where task parameter $\theta \in \Theta_i$ for $i = 0, \ldots, n$ defines the domain randomization at each step in the algorithm. It is pivotal that the learned policy π_θ^i at each cycle has a high success rate; thus, Algorithm 1 is terminated if the success rate of the policy is below a preset threshold. A revised curriculum could be defined to resolve such issues. Finally, the set of successful rollouts $\mathcal{D}$ is updated based on the most recent policy. This ensures that the next cycle of the reinforcement learning has access to successful rollouts.

Algorithm 1 Recursive Reference-Policy Curriculum

Require: Ordered curriculum $\{\Theta_i\}_{i=0}^n$, reference policy μ, and successful rollouts $\mathcal{D}$.

for $i = 0, 1, \ldots, n$ **do**
  $(\pi_\theta^i, b) \leftarrow \textsc{TrainStage}(\mu, \mathcal{D}, \Theta_i)$ ▷ Alg. 2
  if $b = 1$ **then**
    $\mu \leftarrow \pi_\theta^i$
    Replace $\mathcal{D}$ with successful rollouts of μ with task parameters uniformly sampled from Θ_i.
  else
    return Failure.
  end if
end for
return Policy μ.

The applied reinforcement learning framework is based on a modification of Imitation Bootstrapped Reinforcement Learning (IBRL) [37], called DRLR [14] that trains a DRL agent π_θ based on a reference policy μ, as shown in Algorithm 2.

D. Policy Adaptation using Rapid Motor Adaptation

When the task parameters change between executions, it is challenging to train a policy that has a high success rate across the entire parameter space. Thus, Rapid Motor Adaptation (RMA) is used for estimating a latent-space representation of an environment's parameters from a history of observations and actions [34].

RMA is a two-phase process that estimates a latent-space representation of an environment's dynamical parameters from a history of observations and actions [34]. In **phase 1**, an encoder η is trained to obtain a latent representation z_t of the privileged parameters e_t, i.e. $z_t = \eta(e_t)$. The encoder η is trained jointly with the reinforcement learning policy π, where the action is given by

$$a_t = \pi(s_t, z_t). \tag{9}$$

In **Phase 2**, encoder η and policy π are fixed, and an encoder ψ is trained to approximate the latent space variable z_t from a history of observations, i.e.

$$\hat{\boldsymbol{z}}_t = \psi(\boldsymbol{x}_{t-k:t-1}, \boldsymbol{a}_{t-k:t-1}), \tag{10}$$

where k is the observation horizon. The applied reinforcement learning policy is consequently defined as

$$a_t = \pi(s_t, \hat{z}_t). \tag{11}$$

Algorithm 2 DRLR: Within-Stage Training

Require: Reference policy μ, successful-rollout buffer $\mathcal{D}$, parameter set Θ, training budget T, evaluation interval T_{eval}, and success-rate threshold $p_{\text{threshold}}$, warm-up time T_{wa}.

Initialize empty online replay $\mathcal{B}$.
Initialize accept flag $b \leftarrow 0$
Initialize target and critic networks $Q_{\phi_1'}, Q_{\phi_2'}$ and Q_{ϕ_1}, Q_{ϕ_2} respectively.
for $t = 0, 1, 2, \ldots, T-1$ **do**
  if $t < T_{\text{wa}}$ **then**
    Execute $a^{\mu} = \mu(s_t)$
  else
    Perform action selection

$$a^{\pi} \sim \pi_\theta(\cdot \mid s) \quad a^{\mu} = \mu(s),$$
$$g(s) = \mathbf{1}\left[\min_{i \in \{1,2\}} \big(Q_{\phi_i'}(s, a^{\pi}) - Q_{\phi_i'}(s, a^{\mu})\big) > 0\right],$$
$$a^*(s) = g(s)a^{\pi} + \big(1 - g(s)\big)a^{\mu}.$$

    Execute $a^*(s)$
  end if
  Add the resulting transition to $\mathcal{B}$.
  Form a minibatch $\mathcal{M}$ from $\mathcal{B}$ and $\mathcal{D}$.
  Compute the clipped Bellman target:

$$v' = \min_i Q_{\phi_i'}(s', a') - \alpha g' \log \pi_\theta(a^{\pi\prime} \mid s'),$$
$$y = \text{clip}\big(r + \gamma(1-d)v',\, y_{\min}, y_{\max}\big).$$

  Update both critics using $\mathcal{M}$ towards y by minimizing

$$\mathcal{L}_{Q_i} = \text{MSE}(Q_{\phi_i}(s,a),\, y)\,, \qquad i \in \{1,2\}.$$

  if $t > T_{\text{wa}}$ **then**
    Update π_θ and its privileged encoder using the SAC actor objective.
  end if
  Polyak-update the target critics.

$$\phi_i' \leftarrow (1-\tau)\phi_i' + \tau\phi_i, \quad i \in \{1,2\}.$$

  if $t \bmod T_{\text{eval}} = 0$ or $t = T-1$ **then**
    Evaluate the current actor π_θ alone under Θ to obtain $\hat{p}_{\text{success}}$.
    Retain the policy π_θ^* with the highest evaluation success rate so far.
    if $\hat{p}_{\text{success}} \geq p_{\text{threshold}}$ **then**
      $b \leftarrow 1$.
      break
    end if
  end if
end for
return π_θ^*, b.

V. DLO Insertion

The DLO insertion task is formulated as a Markov decision process, where the policy relies on state information from geometric observations of the DLO centerline; however, some task parameters are unmeasured. The policy ensures robustness towards different task parameters by the use of RMA.

To solve the DLO insertion task, the task is divided into two subtasks: alignment and insertion. Alignment involves moving the DLO tip close to the hole center and ensuring that the DLO is aligned with normal vector n. Insertion involves moving the DLO a distance l into the cylinder.

The observation at each sample consists of points along the DLO centerline $\{\gamma_t(v_i)\}_{i=1}^{n_{\text{DLO}}}$, where $v_i \in [0,1]$ for all i, the unit vector $\boldsymbol{z}^{\text{tip}}$, the grasp-frame $\mathcal{F}^{\text{TCP}}$, and the previous action $\boldsymbol{a}_{t-1}$. These observations capture the current kinematics of the system and include the previous action to provide temporal context.

When applying RMA, an additional privileged latent vector $\boldsymbol{z} \in \mathbb{R}^8$ is used in simulation and replaced by its estimate $\hat{\boldsymbol{z}} \in \mathbb{R}^8$ during deployment. The privileged parameter e_t is $(\theta_{DLO}, \theta_{env}, \theta_{init})$ except parameters ρ, ρ_{DLO}, α.

The action space is defined to be a change in the robot pose i.e.

$$a = [\Delta \boldsymbol{p} \quad \Delta \boldsymbol{r}] \in \mathbb{R}^6 \tag{12}$$

where $\Delta \boldsymbol{r}$ is a rotation vector.

The two subtasks each has a termination function. Both terminate when the episode times out ($t \geq T$) and are therefore assumed under both termination functions.

Alignment termination occurs when the tip axis aligns with the tube within a threshold $\overline{r}$ and when the tip position is within a specified positional threshold $\overline{p}$

$$d_{\text{al}} = \begin{cases} \text{True} & \text{if } \|\gamma_t(1) - c\|_2 < \overline{p} \;\wedge\; n^{\top}\boldsymbol{z}^{\text{tip}} > \overline{r}, \\ \text{False} & \text{otherwise.} \end{cases} \tag{13}$$

Insertion terminates when the tip position is inserted a distance l into the cylinder as defined in (2). The insertion termination function is therefore

$$d_{\text{in}} = \begin{cases} \text{True} & \text{if } \gamma_t(1) \in \Omega_{in} \\ \text{False} & \text{otherwise.} \end{cases} \tag{14}$$

Finally, the sparse reward is defined as

$$R_{\text{sparse},t} = \begin{cases} 100 & \text{if } d_{\text{task}} \\ -0.01 & \text{else.} \end{cases} \tag{15}$$

where d_{task} is the successful termination condition for each of the tasks.

VI. Results

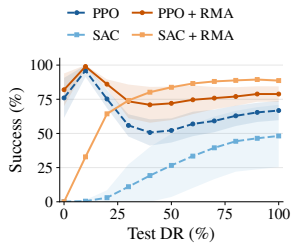
This section presents the results from simulation and experimental validation of the proposed RL framework.

A. Significance of parameter estimation

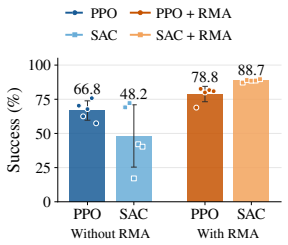
We show the importance of estimating latent DLO parameters by comparing the task success rate of the RL algorithms Proximal Policy Optimization (PPO) and SAC with and without access to privileged parameters as explained in Section IV-D (Tab. IV provides information on domain randomization parameters). The policies are trained with

the dense reward defined in Appendix A and with domain randomization introduced according to Appendix **??**. Fig. 4a shows the success rate of the RL policies during training at different levels of domain randomization.

Fig. 4b shows the success rate of the trained policy evaluated on 1000 rollouts in simulation for five different seeds. The results show that the addition of privileged information significantly improves the performance of both RL algorithms. A dense reward was used for the above analysis, as PPO achieved 100% success without DR, while the sparse reward yielded 0% success under the same conditions. This is the motivation for the introduction of expert-initialized reinforcement learning in the next subsections.



(a) Final-policy success across test DR levels. Shading denotes one standard deviation across runs.



(b) Final-policy success at 100% DR. Bars show means, error bars indicate one standard deviation, and markers show individual runs.

Fig. 4: Insertion success of PPO and Soft Actor-Critic (SAC) with and without privileged Phase-1 RMA. Each method comprises five seeds, evaluated over 1,000 rollouts per run and at the test DR level. PPO and SAC use 1.2 and 2 billion training transitions, respectively.

The PPO+RMA policy is evaluated with different cable lengths. Fig. 5 indicates that generalization is possible for short cables, but not for long cables. This indicates that the task parameters significantly affect performance.

B. DLO Insertion in Simulation

Four DLO insertion strategies are evaluated in simulation by performing 100 rollouts with task parameters selected uniformly from Θ_n (the parameter set defined in Appendix IV) and by performing 100 rollouts with task parameters selected uniformly from the parameter set ϕ defined in Tab. V. The resulting success rates for different DLO insertion strategies are shown in Tab. I.

TABLE I: Success rates on domains ϕ and Φ, comparing [5], DRLR, and recursive DRLR+RMA with a fixed reference policy, fixed latent, or both components updated (ours).

Algorithms	ϕ	Φ
[5]	89%	-
DRLR	-	7.5%
Recursiv DRLR+RMA (fixed μ)	-	2.0%
Recursiv DRLR+RMA (fixed $\hat{z}_t$)	-	10.6%
Recursiv DRLR+RMA (**Ours**)	**100%**	**98.5%**

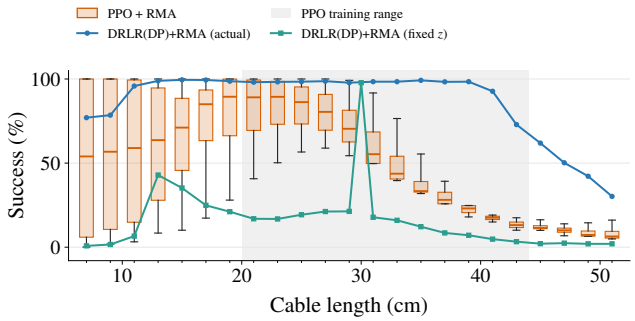


Fig. 5: Insertion success versus cable length for PPO+RMA and DRLR(DP)+RMA, using 1,000 rollouts. The box plot summarizes five PPO+RMA training seeds. Gray shading denotes the PPO training length range (20-44 cm). DRLR(DP)+RMA (actual) is the policy when the actual length is given to the privileged encoder, while DRLR(DP)+RMA (fixed z) denotes the case where the length parsed to the encoder is fixed to 30.

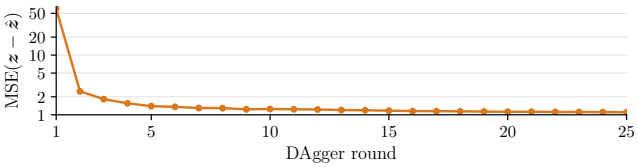


Fig. 6: Phase 2 Dataset Aggregation (DAgger) training loss. Mean squared error between the history encoder's predicted latent and the privileged teacher's latent over 25 DAgger rounds.

After training the policy with privileged encoder η, the Phase 2 encoder is trained using DAgger. As shown in Fig. 6, the MSE reaches 1.1, corresponding to an RMS error of $\sqrt{8 \cdot 1.1} \approx 2.97$, or 5.1% of the reference latent norm (58.6). The resulting policy achieves 96.1% success at 100% DR.

C. Experimental Validation of DLO Insertion

Experimental validation is performed with three different DLOs in the setup shown in Fig. 7. The applied DLO

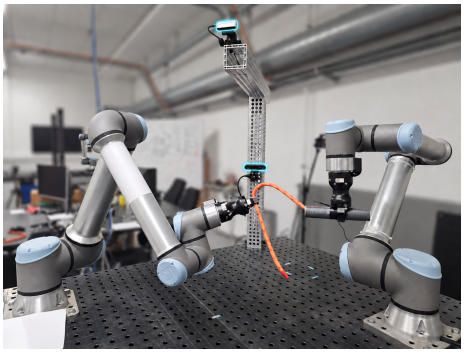


Fig. 7: Setup for real experiments using two RealSense cameras (marked in blue) for multi-view DLO pose estimation from [38].

tracking algorithm was based on the multiview method from [38]. The algorithm was modified to handle a single boundary condition. During insertion, the DLO is in contact with the cylinder. interacts with the inner surface of the tube. To ensure that the estimated configuration remains consistent with the simulated contact geometry, we additionally impose a cylindrical constraint corresponding to the known tube boundary. Nodes that penetrate this boundary are kinematically projected back onto the feasible region, thereby approximating the effect of contact with the tube.

We consider three DLO lengths, $L_1 = 15\,\mathrm{cm}$, $L_2 = 22\,\mathrm{cm}$, and $L_3 = 30\,\mathrm{cm}$, and three material configurations.

TABLE II: Real setup success rates across DLO length and material properties over 10 trials.

Material	Insertion			Alignment		
	L_1	L_2	L_3	L_1	L_2	L_3
Cable	100%	100%	100%	100%	100%	100%
Rope	100%	100%	100%	100%	100%	100%
Cable spiral	100%	100%	100%	100%	100%	100%

VII. CONCLUSION

This paper presented a recursive expert-initialized reinforcement learning that allows the use of sparse rewards due to expert initialization and extensive domain randomization due to recursive update of the reference policy and gradual expansion of task parameter set. Zero-shot transfer of the RL policy to real-world DLO insertion is demonstrated with a success rate of 100% for nine test cases.

Finally, the generality of the proposed framework should be further evaluated by applying the same setup to a wider range of DLO manipulation tasks, including shape control and routing.

REFERENCES

[1] S. Makris, F. Dietrich, K. Kellens, and S. J. Hu, "Automated assembly of non-rigid objects," *CIRP Annals*, vol. 72, no. 2, pp. 513–539, 2023.

[2] S. Makris, E. Kampourakis, and D. Andronas, "On deformable object handling: Model-based motion planning for human-robot comanipulation," *CIRP Annals*, vol. 71, no. 1, pp. 29–32, 2022.

[3] J. Sanchez, J.-A. Corrales, B.-C. Bouzgarrou, and Y. Mezouar, "Robotic manipulation and sensing of deformable objects in domestic and industrial applications: a survey," *The International Journal of Robotics Research*, vol. 37, no. 7, pp. 688–716, 2018.

[4] D. De Gregorio, R. Zanella, G. Palli, S. Pirozzi, and C. Melchiorri, "Integration of robotic vision and tactile sensing for wire-terminal insertion tasks," *IEEE Transactions on Automation Science and Engineering*, vol. 16, no. 2, pp. 585–598, 2019.

[5] M. Li and C. Choi, "Learning for deformable linear object insertion leveraging flexibility estimation from visual cues," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 5183–5189.

[6] M. Li, H. Yu, and C. Choi, "Routing manipulation of deformable linear object using reinforcement learning and diffusion policy," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 1–7.

[7] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Robotics: Science and Systems XIX*. Robotics: Science and Systems Foundation, 2023.

[8] J. Lv, Y. Feng, C. Zhang, S. Zhao, L. Shao, and C. Lu, "SAM-RL: Sensing-aware model-based reinforcement learning via differentiable physics-based simulation and rendering," *The International Journal of Robotics Research*, vol. 44, no. 10–11, pp. 1767–1783, 2025.

[9] M. Li, H. Yu, Y. Huang, Y. Hong, H. Ye, and C. Choi, "Hierarchical DLO routing with reinforcement learning and in-context vision-language models," in *2026 IEEE International Conference on Robotics and Automation (ICRA)*, 2026.

[10] R. P. McKennaa and J. Oyekan, "A perspective on open challenges in deformable object manipulation," 2026.

[11] V. G. Goecks, G. M. Gremillion, V. J. Lawhern, J. Valasek, and N. R. Waytowich, "Integrating behavior cloning and reinforcement learning for improved performance in dense and sparse reward environments," in *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 2020, pp. 465–473.

[12] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, "Efficient online reinforcement learning with offline data," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 1577–1594.

[13] H. Hu, S. Mirchandani, and D. Sadigh, "Imitation bootstrapped reinforcement learning," in *Robotics: Science and Systems XX*. Robotics: Science and Systems Foundation, 2024.

[14] C. Shen and C. Sloth, "Solving robotics tasks with prior demonstration via exploration-efficient deep reinforcement learning," *Frontiers in Robotics and AI*, vol. 12, 2026.

[15] J. Luo, C. Xu, J. Wu, and S. Levine, "Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning," *Science Robotics*, vol. 10, no. 105, p. eads5033, 2025.

[16] P. Kicki, A. Szymko, and K. Walas, "DLOFTBs – fast tracking of deformable linear objects with b-splines," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 7104–7110.

[17] S. Wang, G. Shen, S. Wu, and D. Wu, "Self-supervised learning of reconstructing deformable linear objects under single-frame occluded view," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 16 028–16 034.

[18] K. Lv, M. Yu, S. Wang, X. Ji, and X. Li, "UniStateDLO: Unified generative state estimation and tracking of deformable linear objects under occlusion for constrained manipulation," 2025.

[19] S. Jin, W. Lian, C. Wang, M. Tomizuka, and S. Schaal, "Robotic cable routing with spatial representation," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5687–5694, 2022.

[20] J. Luo, C. Xu, X. Geng, G. Feng, K. Fang, L. Tan, S. Schaal, and S. Levine, "Multistage cable routing through hierarchical imitation learning," *IEEE Transactions on Robotics*, vol. 40, pp. 1476–1491, 2024.

[21] J. Xiang, H. Dinkel, H. Zhao, N. Gao, B. Coltin, T. Smith, and T. Bretl, "TrackDLO: Tracking deformable linear objects under occlusion with motion coherence," *IEEE Robotics and Automation Letters*, vol. 8, no. 10, pp. 6179–6186, 2023.

[22] J. Xiang and H. Dinkel, "MultiDLO: Simultaneous shape tracking of multiple deformable linear objects with global-local topology preservation," in *IEEE International Conference on Robotics and Automation (ICRA) Workshop on Representing and Manipulating Deformable Objects*, May 2023.

[23] H. Dinkel, M. Büsching, A. Longhini, B. Coltin, T. Smith, D. Kragic, M. Björkman, and T. Bretl, "DLO-Splatting: Tracking deformable linear objects using 3d gaussian splatting," in *IEEE International Conference on Robotics and Automation (ICRA), 5th Workshop on Reflections on Representations and Manipulating Deformable Objects*, 2025.

[24] S. Zhaole, J. Zhu, and R. B. Fisher, "DexDLO: Learning goal-conditioned dexterous policy for dynamic manipulation of deformable linear objects," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 16 009–16 015.

[25] J. Ramírez, W. Yu, and A. Perrusquía, "Model-free reinforcement learning from expert demonstrations: a survey," *Artificial Intelligence Review*, vol. 55, no. 4, pp. 3213–3241, 2022.

[26] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," in *Robotics:*

Science and Systems XIV. Robotics: Science and Systems Foundation, 2018.

[27] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine, "SERL: A software suite for sample-efficient robotic reinforcement learning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 16 961–16 969.

[28] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 23–30.

[29] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 3803–3810.

[30] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, 2019.

[31] J. Josifovski, S. Auddy, M. Malmir, J. Piater, A. Knoll, and N. Navarro-Guerrero, "Continual domain randomization," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 4965–4972.

[32] N. Sobanbabu, G. He, T. He, Y. Yang, and G. Shi, "Sampling-based system identification with active exploration for legged sim2real learning," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305. PMLR, 27–30 Sep 2025, pp. 578–598.

[33] J. Gao, M. Y. Michelis, A. Spielberg, and R. K. Katzschmann, "Sim-to-real of soft robots with learned residual physics," *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 8523–8530, 2024.

[34] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Robotics: Science and Systems XVII*. Robotics: Science and Systems Foundation, 2021.

[35] A. Kumar, Z. Li, J. Zeng, D. Pathak, K. Sreenath, and J. Malik, "Adapting rapid motor adaptation for bipedal robots," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 1161–1168.

[36] R. Chen, M. Mukadam, M. Kaess, T. Wu, F. R. Hogan, J. Malik, and A. Sharma, "PTLD: Sim-to-real privileged tactile latent distillation for dexterous manipulation," 2026.

[37] Z.-H. Yin, W. Ye, Q. Chen, and Y. Gao, "Planning for sample efficient imitation learning," in *Advances in Neural Information Processing Systems 35*. Neural Information Processing Systems Foundation, Inc., 2022, pp. 2577–2589.

[38] A. Caporali and G. Palli, "Robotic manipulation of deformable linear objects via multiview model-based visual tracking," *IEEE/ASME Transactions on Mechatronics*, vol. 30, no. 5, pp. 3966–3977, 2025.

APPENDIX

TABLE III: Constants and parameters used in this work.

Parameter	Symbol	Value
Training time steps	T_{train}	300 000
Warmup time steps	T_{warmup}	2 000
Termination position threshold	$\bar{p}$	0.02 m
Termination rotation threshold	$\bar{r}$	10°
Episode length	T	1000
Dense reward position gain	w_p	1
Dense reward rotation gain	w_o	1
Reward done gain	w_d	25
Sparse reward scale	w_s	10 000
DP observation horizon	H_o	2
DP prediction horizon	H_p	8
DP action horizon	H_a	4
Exponential Moving Average (EMA) weight	α	0.75
Number of DLO pts.	n_{dlo}	10
Weight for desired homotopy class	w_h^+	10
Weight for undesired homotopy class	w_h^-	10
Number of DLO pts.	n_{dlo}	10
Diffusion iterations	n_k	50

TABLE IV: List of task parameters and their full parameter ranges.

Parameter	DR range	Unit
DLO Young's modulus	[0.10, 10.00]	MPa
DLO shear modulus	[0.10, 10.00]	MPa
DLO radius	[4.0, 8.2]	mm
DLO length	[150, 350]	mm
DLO linear density	[4.375, 17.5]	$\mathrm{g\,m^{-1}}$
DLO bending damping	[0.0025, 0.040]	–
DLO–cylinder sliding friction	[0.20, 1.50]	–
Radius of cylinder	[10, 20]	mm
Cylinder length	[100, 130]	mm
Angle of cylinder	[−90, 0]	deg
Initial-curvature magnitude	[0, 60]	deg
Initial-curvature direction	[0, 360]	deg
DLO and tip position error, RMS	[0, 10]	mm
Tip-direction error, RMS	[0, 5]	deg
TCP position error, RMS	[0, 0.5]	mm
TCP orientation error, RMS	[0, 0.25]	deg
Cylinder-frame position error, RMS	[0, 1.0]	mm
Cylinder-frame orientation error, RMS	[0, 0.5]	deg

A. Dense Reward Function

The dense reward is defined as:

$$R_t = E_{t-1} - E_t + R_{\text{homo},t} + R_{\text{done},t}, \tag{16}$$

$$E_t = w_p \frac{p_t}{\max(\bar{p}, \varepsilon)} + w_o \frac{r_t}{\max(\bar{r}, \varepsilon)}, \tag{17}$$

where p_t and r_t are the position and orientation errors, $\bar{p}$ and $\bar{r}$ their success thresholds, w_p, w_o weights, and $\varepsilon = 10^{-6}$.

Geometric shaping penalizes tip positions outside the admissible cylinder extended along its unit axis n:

$$R_{\text{homo},t} = -w_h \, \mathbf{1}\{\gamma_t(1) \notin \mathcal{H}\}, \tag{18}$$

$$\mathcal{H} = \left\{ x \in \mathbb{R}^3 : \left\| (I - nn^\top)(x - c) \right\|_2 \le \rho - \rho_{\text{DLO}} \right\}, \tag{19}$$

where c lies on the cylinder axis and ρ and ρ_{DLO} denote the tube's inner radius and cable radius.

Finally,

$$R_{\text{done},t} = \begin{cases} +w_s, & \text{task success,} \\ -w_f, & t \ge T \text{ without success,} \\ 0, & \text{otherwise,} \end{cases} \tag{20}$$

with success defined by the alignment or insertion criterion.

The curriculum is introduced as

B. ϕ Curriculum

TABLE V: Randomized task parameters ϕ

Parameter	DR range	Unit
Radius of cylinder	[10, 25]	mm
Angle of cylinder	[−90, 0]	deg
DLO Young's modulus	[4.00, 6.00]	MPa
DLO Shear modulus	[4.00, 6.00]	MPa